CISS245: Advanced Programming
Quiz q8001

Name: eagraham1@cougars.ccis.edu Score:

Q1. In the code below, there are common member variables in `TeslaOptimusRobot` and `RoombaRobot`. The goal is to clean up the repetitive code by moving the common parts of these two classes into a parent class.

- Complete the `Robot` class with the common member variables of `TeslaOptimusRobot` and `Roomba` in its private section.
- In the public section, write constructor for `Robot` to initialize these member variables and write public constant methods `Robot::get_name()`, `Robot::get_x()`, and `Robot::get_y()`.
- Make `TeslaOptimusRobot` and `RoombaRobot` subclasses of `Robot` using public inheritance. Modify `TeslaOptimusRobot` and `RoombaRobot` accordingly.
- There should be no change to `main()` and the completed code should run the same as before.

Answer:

```
#include <iostream>
#include <string>

class Robot
{
public:
Robot(std::string name, int x, int y)
: name_(name), x_(x), y_(y)
{}
std::string get_name() const
{
  return name_;
}
int get_x() const
{
  return x_;
}
int get_y() const
{
  return y_;
}
private:
std::string name_;
int x_, y_;
```

```
};

class TeslaOptimusRobot: public Robot
{
public:
    TeslaOptimusRobot(std::string name, int x, int y, int num_legs)
        : Robot(name, x, y), num_legs_(num_legs)
    {}
    void print() const
    {
        std::cout << "<TeslaOptimusRobot "
                  << "name:" << name_ << ' '
                  << "x:" << get_x() << ' '
                  << "y:" << get_y() << ' '
                  << "num_legs:" << num_legs_
                  << ">\n";
    }
    void talk() const
    {
        std::cout << "<TeslaOptimusRobot name:"
                  << get_name()
                  << "> says \"hello world\"\n";
    }
private:
    int num_legs_;
};

class RoombaRobot: public Robot
{
public:
    RoombaRobot(std::string name, int x, int y, int num_wheels)
        : Robot(name, x, y), num_wheels_(num_wheels)
    {}
    void print() const
    {
        std::cout << "<RoombaRobot "
                  << "name:" << get_name() << ' '
                  << "x:" << get_x() << ' '
                  << "y:" << get_y() << ' '
                  << "num_wheels:" << num_wheels_
                  << ">\n";
    }
    void talk() const
    {
        std::cout << "<RoombaRobot name:"
                  << get_name()
                  << "> says \"whirrrr\"\n";
    }
private:
    int num_wheels_;
};
```

```
int main()
{
    TeslaOptimusRobot arnold("arnold", 0, 0, 2);
    RoombaRobot neo("i_am_the_chosen_one", 5, 0, 3);

    arnold.print();
    neo.print();

    arnold.talk();
    neo.talk();

    return 0;
}
```

Instructions

In the file `thispreamble.tex` look for

```
\renewcommand\AUTHOR{}
```

and enter your email address:

```
\renewcommand\AUTHOR{jdoe5@cougars.ccis.edu}
```

(This is not really necessary since alex will change that for you when you execute `make`.) In your bash shell, execute "`make`" to recompile `main.pdf`. Execute "`make v`" to view `main.pdf`.

Enter your answers in `main.tex`. In the bash shell, execute "`make`" to recompile `main.pdf`. Execute "`make v`" to view `main.pdf`.

For each question, you'll see boxes for you to fill. For small boxes, if you see

```
1 + 1 = \answerbox{}.
```

you do this:

```
1 + 1 = \answerbox{2}.
```

`answerbox` will also appear in "true/false" and "multiple-choice" questions.

For longer answers that need typewriter font, if you see

```
Write a C++ statement that declares an integer variable name x.
\begin{answercode}
\end{answercode}
```

you do this:

```
Write a C++ statement that declares an integer variable name x.
\begin{answercode}
int x;
\end{answercode}
```

`answercode` will appear in questions asking for code, algorithm, and program output. In this case, indentation and spacing is significant. For program output, I do look at spaces and newlines.

For long answers (not in typewriter font) if you see

```
What is the color of the sky?
\begin{answerlong}
\end{answerlong}
```

you can write

```
What is the color of the sky?
\begin{answerlong}
The color of the sky is blue.
\end{answerlong}
```

A question that begins with "T or F or M" requires you to identify whether it is true or false, or meaningless. "Meaningless" means something's wrong with the question and it is not well-defined. Something like "$1 + 2 = 4$" is either true or false (of course it's false). Something like "$1+_2 = 4$?" does not make sense.

When writing results of computations, make sure it's simplified. For instance write 2 instead of $1 + 1$.

Higher level classes.

For students beyond 245: You can put LaTeX commands in `answerlong`.

More examples of meaningless statements: Questions such as "Is $42 = 1+_2$ true or false?" or "Is $42 = \{2\}^{\{3\}}$ true or false?" does not make sense. "Is $P(42) = \{42\}$ true or false?" is meaningless because $P(X)$ is only defined if X is a set. For "Is $1 + 2 + 3$ true or false?", "$1 + 2 + 3$" is well–defined but as a "numerical expression", not as a "proposition", i.e., it cannot be true or false. Therefore "Is $1 + 2 + 3$ true or false?" is also not a well-defined question.

More examples of simplification: When you write down sets, if the answer is $\{1\}$, do not write $\{1, 1\}$. And when the values can be ordered, write the elements of the set in ascending order. When writing polynomials, begin with the highest degree term.

When writing a counterexample, always write the simplest.